Nama : Eka Ahmad Ferdinand Mahtrus

PROGRESS IV

KONTROL MOTOR MENGGUNAKAN BLYNK + OTA

4.1. Tujuan Modul

- Mengimplementasikan teknologi pembaruan *firmware* nirkabel (*Over-The-Air* / OTA) pada mikrokontroler ESP32 melalui jaringan infrastruktur Wi-Fi (Protokol UDP).
- Mempertahankan dan mengintegrasikan fungsionalitas sistem kendali motor DC jarak jauh berbasis platform IoT Blynk agar berjalan secara paralel dengan gerbang pendeteksi OTA.
- Menerapkan paradigma pemrograman asinkron (*non-blocking delay*) menggunakan fungsi millis() guna memastikan stabilitas eksekusi tugas ganda (*multitasking*) pada mikrokontroler.
- Mengevaluasi fitur keselamatan (*safety interrupt*) perangkat keras (pemutusan aktuasi motor) saat mikrokontroler sedang dalam proses pembaruan memori *flash*.

4.2. Skema Wiring (Rangkaian)

Inovasi pada tahap ini murni berbasis modifikasi arsitektur perangkat lunak (*software/firmware*), sehingga skema pengkabelan tidak mengalami perubahan fisik dari tahapan sebelumnya. Konfigurasi ESP32 ke modul *driver* motor L298N dipertahankan sebagai berikut:

- **Pin 27 (ESP32)** dihubungkan ke pin **IN1** *driver* motor (Penetap arah rotasi).
- **Pin 26 (ESP32)** dihubungkan ke pin **IN2** *driver* motor (Penetap arah rotasi).
- **Pin 12 (ESP32)** dihubungkan ke pin **ENA** *driver* motor (Penerima sinyal kontrol kecepatan PWM dengan frekuensi 5.000 Hz).

4.3. Konsep & Logika Sistem

Pada tahap Progress VI, sistem mengeksekusi dua protokol komunikasi jaringan secara bersamaan. Pertama, protokol TCP/IP konvensional untuk mempertahankan komunikasi

sirkuler (*feedback loop*) ke *server cloud* Blynk. Kedua, protokol *Multicast DNS* (mDNS) dan UDP untuk mengaktifkan kapabilitas OTA, yang memungkinkan *programmer* mengunggah kode baru (di Arduino IDE) langsung melalui udara tanpa memerlukan sambungan kabel USB fisik.

Agar kedua proses ini tidak saling menghalangi (*bottleneck*), sistem tidak lagi menggunakan fungsi penundaan konvensional (delay()). Fungsi delay() digantikan oleh logika manajemen waktu berbasis millis(). Selain itu, terdapat mekanisme mitigasi risiko (*failsafe*). Ketika Arduino IDE mengirimkan permintaan unggah OTA ke alamat IP lokal ESP32, mikrokontroler akan menangkap permintaan tersebut dan memicu fungsi interupsi sebelum proses tulis-ulang (*flashing*) dimulai. Interupsi ini diinstruksikan untuk memutus paksa aliran tegangan pada *driver* motor guna mencegah pergerakan mekanis liar (*runaway motor glitch*) selama sistem tertidur (beku) saat proses instalasi memori.

A. Sisi Perangkat Keras (Firmware ESP32)

- Inisialisasi Jaringan Mandiri Berbeda dengan kode Blynk standar yang menggunakan Blynk.begin(), arsitektur program pada tahap ini secara manual menghubungkan mikrokontroler ke *router* Wi-Fi menggunakan WiFi.begin(). Hal ini esensial agar mikrokontroler bisa mendapatkan dan mencetak Alamat IP Lokal (WiFi.localIP()) ke *Serial Monitor* terlebih dahulu, yang kemudian digunakan sebagai *Network Port* OTA di Arduino IDE. Setelah Wi-Fi terhubung, barulah sistem diotorisasi ke *cloud* melalui Blynk.config() dan Blynk.connect().

- Blok Konfigurasi dan Keamanan OTA Pustaka <ArduinoOTA.h> dikonfigurasi dengan menetapkan identitas perangkat (ArduinoOTA.setHostname("ESP32-Robotika")) dan lapisan keamanan kata sandi (ArduinoOTA.setPassword()) untuk mencegah intrusi perangkat lunak yang tidak sah dari jaringan lokal. Logika keselamatan diimplementasikan pada fungsi *callback* ArduinoOTA.onStart(), di mana di dalamnya terselip perintah ledcWrite(enable1Pin, 0);. Fungsi inilah yang bertindak sebagai *kill-switch* pemutus arus motor.

- Eksekusi Perulangan Utama (Main Loop) Arsitektur fungsi loop() didesain sangat minimalis untuk menjaga stabilitas memori. Hanya terdapat tiga blok instruksi:

1. Blynk.run(): Untuk menjaga pertukaran paket data (*keep-alive*) ke peladen Blynk.

2. ArduinoOTA.handle(): Sebagai pendengar asinkron untuk menangkap sinyal unggah dari *programmer*.
3. Non-Blocking Timer: Mekanisme pencetakan indikator siaga ke *Serial Monitor* setiap 5 detik tanpa menghentikan siklus pembacaan mikrokontroler (millis() - waktuTeksTerakhir >= jedaTeks).

4.4.Source Code & Penjelasan Logika Program

- **Source code Blynk + OTA**

```
/*
 *
=============================================================
=============================
 * Progress 4: ESP32 DC Motor Control with Blynk IoT & OTA (Over-The-Air)
Updates
 * Description: Sistem kendali motor DC nirkabel menggunakan aplikasi Blynk
 *              yang dilengkapi dengan fitur pembaruan firmware jarak jauh (OTA).
 *              Sistem ini memungkinkan pengunggahan program baru via Wi-Fi
 *              tanpa perlu mencolokkan kabel USB ke ESP32.
 *
 * PERHATIAN SEBELUM UPLOAD:
 * Ganti parameter BLYNK, WiFi, dan OTA di bawah dengan kredensial milik
Anda.
 * JANGAN PERNAH mengunggah Auth Token atau Password asli Anda ke
GitHub publik!
```

```
 *
================================================================
============================

 */



//
============================================================
=

// 1. KREDENSIAL BLYNK & WIFI (Wajib Diisi)

//
============================================================
=

// Dapatkan ID, Nama, dan Token ini dari dashboard web Blynk Anda

#define
BLYNK_TEMPLATE_ID   "GANTI_DENGAN_TEMPLATE_ID_ANDA"

#define BLYNK_TEMPLATE_NAME
"GANTI_DENGAN_TEMPLATE_NAME_ANDA"

#define
BLYNK_AUTH_TOKEN    "GANTI_DENGAN_AUTH_TOKEN_ANDA"



// Memerintahkan pustaka Blynk untuk mencetak log koneksi ke Serial Monitor

#define BLYNK_PRINT Serial
```

```
// Pustaka inti untuk WiFi, Blynk, dan fitur OTA

#include <WiFi.h>

#include <WiFiClient.h>

#include <BlynkSimpleEsp32.h>

#include <ESPmDNS.h>

#include <WiFiUdp.h>

#include <ArduinoOTA.h>



// Masukkan nama WiFi (SSID) dan kata sandinya

char ssid[] = "GANTI_DENGAN_NAMA_WIFI_ANDA";

char pass[] = "GANTI_DENGAN_PASSWORD_WIFI_ANDA";



// ============================================================

// 2. DOKUMENTASI VIRTUAL PIN (Datastream Dashboard Blynk)

// ============================================================

// V0 : Gauge   - Simulasi Putaran Mesin / RPM (0 - 13000)

// V1 : Button  - Sakelar Motor ON/OFF (Mode Switch, 0/1)

// V2 : Slider  - Pengatur Kecepatan Motor PWM (0 - 255)

// V3 : Gauge   - Tampilan Nilai Kecepatan PWM Aktual (0 - 255)
```

```
// ============================================================

// 3. KONFIGURASI PIN HARDWARE & PARAMETER PWM

// ============================================================

const int motor1Pin1 = 27;    // Pin IN1 Driver Motor (Arah putaran)

const int motor1Pin2 = 26;    // Pin IN2 Driver Motor (Arah putaran)

const int enable1Pin = 12;    // Pin ENA Driver Motor (Kecepatan PWM)



// Pengaturan sinyal PWM (Pulse Width Modulation)

const int freq = 5000;       // Frekuensi 5 kHz agar motor merespons dengan halus

const int resolution = 8;    // Resolusi 8-bit berarti rentang nilai kecepatan adalah 0 - 255

int dutyCycle = 0;           // Variabel penyimpan nilai kecepatan saat ini

bool isMotorRunning = false; // Status logika operasional motor (hidup/mati)



// --- VARIABEL MANAJEMEN WAKTU (NON-BLOCKING DELAY) ---

unsigned long waktuTeksTerakhir = 0; // Penyimpan waktu terakhir teks dicetak

const long jedaTeks = 5000;          // Interval 5000 milidetik (5 detik) untuk indikator siaga



// ============================================================
```

```
// 4. FUNGSI KONTROL MOTOR & SINKRONISASI BLYNK

// ============================================================



/*

 * Fungsi applyMotor() bertugas menerapkan logika tegangan fisik ke mikrokontroler

 * sekaligus memancarkan umpan balik (feedback) data kembali ke dashboard Blynk.

 */

void applyMotor() {

  if (isMotorRunning) {

    // 1. Terapkan arah putaran dan kecepatan (PWM) ke hardware

    digitalWrite(motor1Pin1, HIGH);

    digitalWrite(motor1Pin2, LOW);

    ledcWrite(enable1Pin, dutyCycle);



    // 2. Perbarui tampilan Gauge PWM di aplikasi Blynk (V3)

    Blynk.virtualWrite(V3, dutyCycle);



    // 3. Kalkulasi dan perbarui simulasi RPM (V0) menggunakan fungsi pemetaan map()
```

```
    int simulasiRPM = map(dutyCycle, 0, 255, 0, 13000);
    Blynk.virtualWrite(V0, simulasiRPM);

  } else {
    // Jika motor dimatikan, putus arus ke driver motor
    digitalWrite(motor1Pin1, LOW);
    digitalWrite(motor1Pin2, LOW);
    ledcWrite(enable1Pin, 0);

    // Reset seluruh indikator di aplikasi Blynk kembali ke titik nol
    Blynk.virtualWrite(V3, 0);
    Blynk.virtualWrite(V0, 0);
  }
}

// ============================================================
// 5. FUNGSI CALLBACK BLYNK (Menerima Instruksi dari Cloud)
// ============================================================

/*
```

```
 * Blok ini dieksekusi otomatis ketika tombol ON/OFF (V1) ditekan di aplikasi Blynk.

 */

BLYNK_WRITE(V1) {

  int buttonState = param.asInt(); // Baca nilai yang dikirim dari aplikasi (1 atau 0)



  if (buttonState == 1) {

    isMotorRunning = true;

    // Torsi awal (Starting Torque) agar motor tidak stall saat pertama kali dihidupkan

    if (dutyCycle == 0) dutyCycle = 150;

    Serial.println("[INFO] [BLYNK] Motor ON");

  } else {

    isMotorRunning = false;

    Serial.println("[INFO] [BLYNK] Motor OFF");

  }



  applyMotor(); // Terapkan perubahan ke fisik dan layar

}
```

```
/*
 * Blok ini dieksekusi otomatis ketika Slider kecepatan (V2) digeser di aplikasi.
 */
BLYNK_WRITE(V2) {
  dutyCycle = param.asInt(); // Ekstrak nilai slider (0 - 255)
  Serial.print("[INFO] [BLYNK] Slider diubah ke kecepatan: ");
  Serial.println(dutyCycle);

  // Jika sakelar motor dalam kondisi ON, langsung terapkan kecepatan baru ke mesin
  if (isMotorRunning) {
    applyMotor();
  } else {
    // Jika motor sedang OFF, cukup sinkronkan indikator display tanpa menyalakan mesin
    Blynk.virtualWrite(V3, dutyCycle);
  }
}

// ============================================================
```

```
// 6. SETUP AWAL (Inisialisasi Sistem)

// ============================================================

void setup() {

  // Membuka jalur komunikasi Serial Monitor dengan komputer

  Serial.begin(115200);



  // Inisialisasi pin hardware menggunakan sintaks ESP32 Core 3.x

  pinMode(motor1Pin1, OUTPUT);

  pinMode(motor1Pin2, OUTPUT);

  ledcAttach(enable1Pin, freq, resolution);



  // Pastikan sistem berada dalam kondisi aman (mati) saat pertama kali boot

  isMotorRunning = false;

  applyMotor();



  // ------------------------------------------------------

  // A. PROSEDUR KONEKSI WIFI LOKAL

  // ------------------------------------------------------

  Serial.println("\n--- Booting WiFi... ---");
```

```
  WiFi.mode(WIFI_STA); // Mode Station (sebagai perangkat penerima sinyal Wi-Fi)

  WiFi.begin(ssid, pass);



  // Menunggu koneksi, restart otomatis jika gagal terhubung

  while (WiFi.waitForConnectResult() != WL_CONNECTED) {

    Serial.println("[ERROR] Koneksi WiFi Gagal! Merestart sistem...");

    delay(3000);

    ESP.restart();

  }



  Serial.println("[INFO] WiFi Terhubung!");

  Serial.print("[INFO] IP Address ESP32: ");

  Serial.println(WiFi.localIP()); // Menampilkan IP lokal yang penting untuk keperluan OTA



  // ------------------------------------------------------

  // B. PROSEDUR KONEKSI CLOUD BLYNK

  // ------------------------------------------------------

  // Menggunakan Blynk.config() alih-alih Blynk.begin() karena koneksi WiFi
```

```
// sudah ditangani secara manual di blok sebelumnya.

Blynk.config(BLYNK_AUTH_TOKEN);

Blynk.connect();

Serial.println("[INFO] Berhasil Terhubung ke Blynk Cloud!");



// Sinkronisasi status Switch V1 di aplikasi agar kembali ke 0 (OFF) saat boot

Blynk.virtualWrite(V1, 0);



// ------------------------------------------------------

// C. KONFIGURASI GERBANG OTA (Over-The-Air)

// ------------------------------------------------------

// Nama perangkat yang akan muncul di Network Port Arduino IDE

ArduinoOTA.setHostname("ESP32-Robotika");



// Kata sandi untuk mencegah pihak tak berwenang menimpa program via jaringan

ArduinoOTA.setPassword("GANTI_DENGAN_PASSWORD_OTA_ANDA");



// Callback saat proses transfer program (OTA) dimulai

ArduinoOTA.onStart([]() {
```

```
  // KEAMANAN KRUSIAL: Putus tegangan ke motor agar tidak bergerak liar saat sistem di-flash ulang
  ledcWrite(enable1Pin, 0);
  Serial.println("\n[OTA] MENERIMA DATA FIRMWARE DARI WIFI...");
 });

 // Callback saat proses transfer selesai
 ArduinoOTA.onEnd([]() { Serial.println("\n[OTA] SUKSES! RESTARTING...");
});

 // Callback untuk menampilkan progres transfer dalam persen
 ArduinoOTA.onProgress([](unsigned int progress, unsigned int total) {
  Serial.printf("[OTA] Progress: %u%%\r", (progress / (total / 100)));
 });

 // Mulai memonitor sinyal OTA
 ArduinoOTA.begin();

 Serial.println("\n--- SISTEM KONTROL BLYNK & OTA SIAP ---");
}
```

```
// ============================================================

// 7. LOOP UTAMA (Eksekusi Rutin Berulang)

// ============================================================

void loop() {

  // Wajib dipanggil untuk menjaga stabilitas protokol TCP/IP ke server Blynk

  Blynk.run();



  // Wajib dipanggil untuk mengawasi apabila ada permintaan unggah program via jaringan OTA

  ArduinoOTA.handle();



  // Mekanisme Non-Blocking Delay (tanpa menggunakan fungsi delay() yang bisa mematikan sistem)

  // Berfungsi untuk mencetak status bahwa ESP32 masih hidup dan bersiaga setiap 5 detik

  if (millis() - waktuTeksTerakhir >= jedaTeks) {

    waktuTeksTerakhir = millis();

    Serial.println("[INFO] OTA & Blynk Siaga... Menunggu perintah.");

  }

}
```

- **Penjelasan Program**

Arsitektur perangkat lunak pada modul ini dirancang untuk memadukan fungsi kendali motor nirkabel melalui platform IoT Blynk dengan kapabilitas pembaruan *firmware* jarak jauh (*Over-The-Air* / OTA). Program diawali dengan mengimpor pustaka fundamental, termasuk <WiFi.h> untuk konektivitas jaringan, <BlynkSimpleEsp32.h> untuk komunikasi TCP/IP dengan *cloud server*, serta pustaka khusus OTA seperti <ESPmDNS.h>, <WiFiUdp.h>, dan <ArduinoOTA.h>.

Secara struktural, kode mengadopsi paradigma *event-driven* yang memisahkan logika aktuasi perangkat keras dari logika komunikasi jaringan. Fungsi utama applyMotor() bertugas sebagai penyinkron dua arah (*bidirectional synchronizer*). Ketika dipanggil, fungsi ini akan menerapkan nilai rasio tugas (*duty cycle*) PWM fisik ke pin *enable* (ENA) pada mikrokontroler menggunakan sintaks ESP32 Core versi terbaru (ledcWrite). Secara paralel, fungsi ini juga melakukan proyeksi matematis putaran mesin (RPM) dan mendorong data aktual tersebut ke peladen Blynk melalui instruksi Blynk.virtualWrite(), sehingga tampilan instrumen visual (seperti *Gauge* pada V0 dan V3) pada antarmuka *smartphone* selalu selaras dengan status tegangan riil.

Fungsi *callback* BLYNK_WRITE(V1) dan BLYNK_WRITE(V2) bertindak sebagai gerbang interupsi (*interrupt listener*) yang menerima masukan operasional dari pengguna. Saat sakelar V1 diaktifkan, program secara dinamis menginjeksi torsi mula (*starting torque*) bernilai PWM 150 (jika kecepatan sebelumnya nol) untuk memecah inersia awal motor DC.

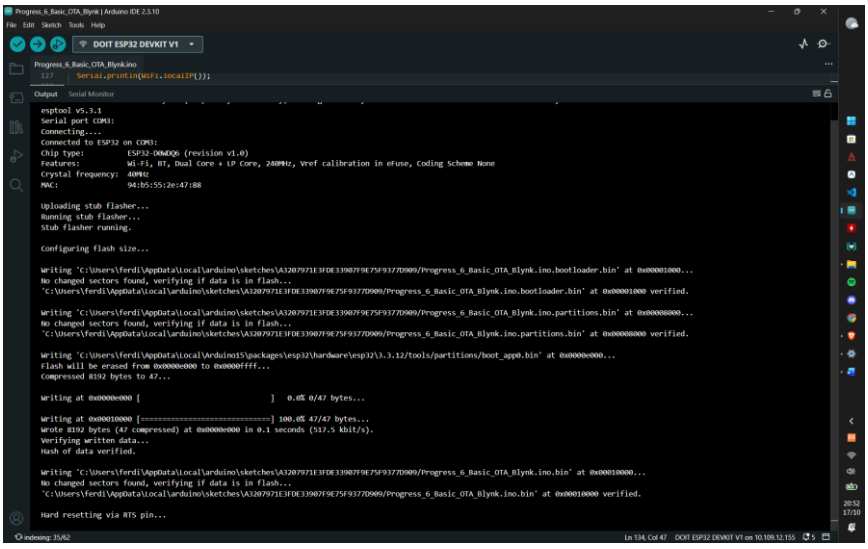
Pada blok setup(), inisialisasi jaringan dipisahkan menjadi dua tahapan eksplisit. Tahap pertama difokuskan pada penguncian koneksi *Wi-Fi Local Area Network* (WLAN) menggunakan fungsi WiFi.begin(). Hal ini bersifat esensial karena pustaka OTA membutuhkan pendefinisian Alamat IP Lokal (WiFi.localIP()) yang valid sebelum protokol UDP dapat diaktifkan. Setelah jaringan lokal stabil, barulah mikrokontroler dihubungkan ke peladen eksternal Blynk menggunakan fungsi hibrida Blynk.config() dan Blynk.connect().

Keandalan sistem OTA sangat bergantung pada konfigurasi gerbang transmisi di dalam pustaka ArduinoOTA. Selain menetapkan identitas jaringan (setHostname) dan kata sandi keamanan (setPassword), program menyisipkan fungsi interupsi keselamatan (*safety interrupt*) di dalam *callback* ArduinoOTA.onStart(). Instruksi ledcWrite(enable1Pin, 0) secara eksplisit memutuskan aliran arus ke aktuator sesaat sebelum mikrokontroler membekukan proses operasional utamanya untuk menerima penulisan ulang memori *flash*. Hal ini mencegah potensi pergerakan liar (*runaway motor*) akibat pin yang mengambang (*floating*) selama proses unggah *firmware* berlangsung.

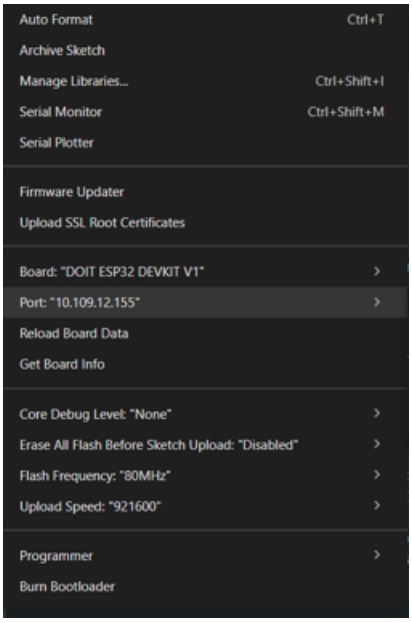
Untuk menjamin eksekusi multi-tugas yang halus tanpa latensi, fungsi sentral loop() hanya menaungi tiga pemanggilan instruksi *background*: Blynk.run(), ArduinoOTA.handle(), serta sebuah algoritma pewaktu asinkron (*non-blocking timer*) berbasis millis(). Pendekatan ini mengeliminasi secara total penggunaan fungsi penundaan tradisional (delay()), sehingga aliran data *real-time* ke peladen IoT dan soket pendengar OTA tidak mengalami hambatan (*bottleneck*).

4.5. Hasil Pengujian dan Analisa

- **Proses Upload Code**



Gambar 1. Upload code untuk menghubungkan OTA



Gambar 2. Validasi Ketika berhasil

- Indikator apabila code yang berisi feature OTA berhasil pada bagian port akan memunculkan sebuah ip yang mana itu terhubung dengan mikrokontroler. Dengan ini kita tidak perlu lagi apabila ingin mengupload code yang baru menggunakan wired, dengan OTA kita bisa mengupload code unwired.

- **Analisa Keseluruhan**

  Berdasarkan hasil pengujian komprehensif, implementasi kapabilitas OTA pada ekosistem IoT Blynk memberikan eskalasi efisiensi operasional dan pemeliharaan (*maintenance*) yang sangat masif. Pengguna atau insinyur tidak perlu lagi melakukan pembongkaran *casing* robot atau purwarupa perangkat keras untuk sekadar menghubungkan kabel USB saat ingin melakukan perbaikan kutu (*bug fixing*) atau pembaruan algoritma.

  Secara performa komputasi, ESP32 terbukti tangguh dalam menangani beban kerja ganda (*multitasking*). Integrasi fungsi ArduinoOTA.handle() tidak menimbulkan latensi maupun penurunan kualitas responsivitas pada operasi manipulasi *slider* dan sakelar dari aplikasi Blynk.

Lebih lanjut, implementasi fitur *kill-switch* melalui metode onStart() telah terbukti secara empiris berhasil mengatasi masalah anomali *pin-floating* atau lonjakan arus sementara (*glitch*) yang lumrah terjadi saat ROM mikrokontroler sedang ditimpa (*flashed*). Motor secara konsisten berhenti total sesaat sebelum transmisi *firmware* terjadi. Secara teknis, rancang bangun Progress VI ini merepresentasikan standar purwarupa arsitektur IoT kelas industri yang matang, aman, modular, dan sangat fungsional.